

Dokumentation Einführung in Game Engines

Space Dodge



Leonard Mottke

m30904

Projektbeschreibung:

Bei diesem Spiel geht es darum, gegnerischen Geschossen auszuweichen. Der Spieler hat eine Lebensanzeige, die mit jedem Treffer kleiner wird. Solange man nicht getroffen wird, erhöht sich ein Zähler. Je länger man durchhält, desto höher wird der Punktestand. Wenn die Lebensanzeige leer ist, kann man das Spiel neustarten, beenden oder zum Hauptmenü zurückkehren. Während des Spiels kann man es auch mit einem Button pausieren.

Projektziel:

Das Ziel war es, ein funktionierendes Spiel mit Unity zu erstellen. Ich habe Unity anstatt Unreal verwendet, um mich intensiver mit dem Schreiben von Skripten auseinanderzusetzen und meine Kenntnisse im Umgang mit dem Programm zu vertiefen.

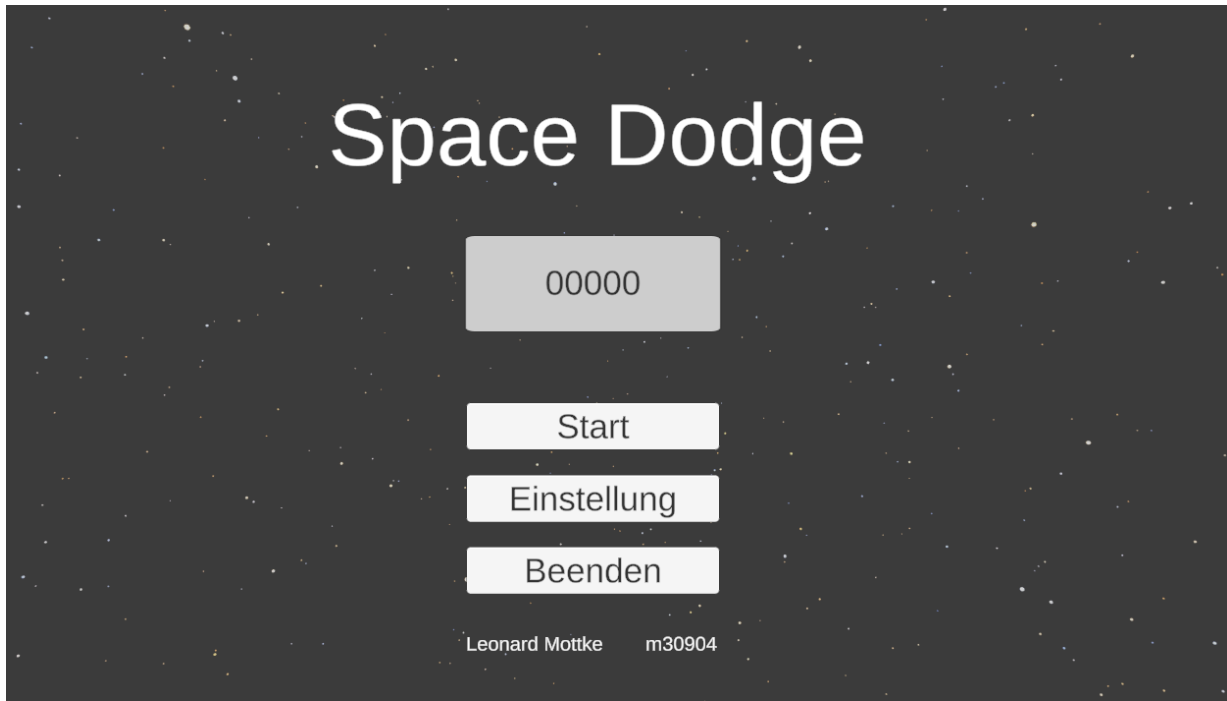
Projektdurchführung:

Zuerst habe ich einen Stichpunktplan erstellt, in dem festgehalten wurde, welche Aufgaben ich nacheinander abarbeite. Als erstes habe ich die Anfangsszene erstellt und mit der Hauptszene verknüpft. Ich habe die verschiedenen Funktionen dafür geschrieben, damit die Buttons funktionieren. Als nächstes habe ich mich um das Gameplay gekümmert. Ich habe das große Raumschiff animiert, sodass es einen bestimmten Pfad entlangfährt. Danach habe ich das Skript für den Spieler, das kleine Raumschiff, geschrieben, damit es sich in einem bestimmten Bereich bewegen kann. Das große Raumschiff hat dann seinen Laser und die Schussfunktion bekommen. Zum Schluss wurden die Musik und ein Soundeffekt eingefügt sowie ein Regler im Hauptmenü, um die Lautstärke von Musik und Effekten anzupassen.

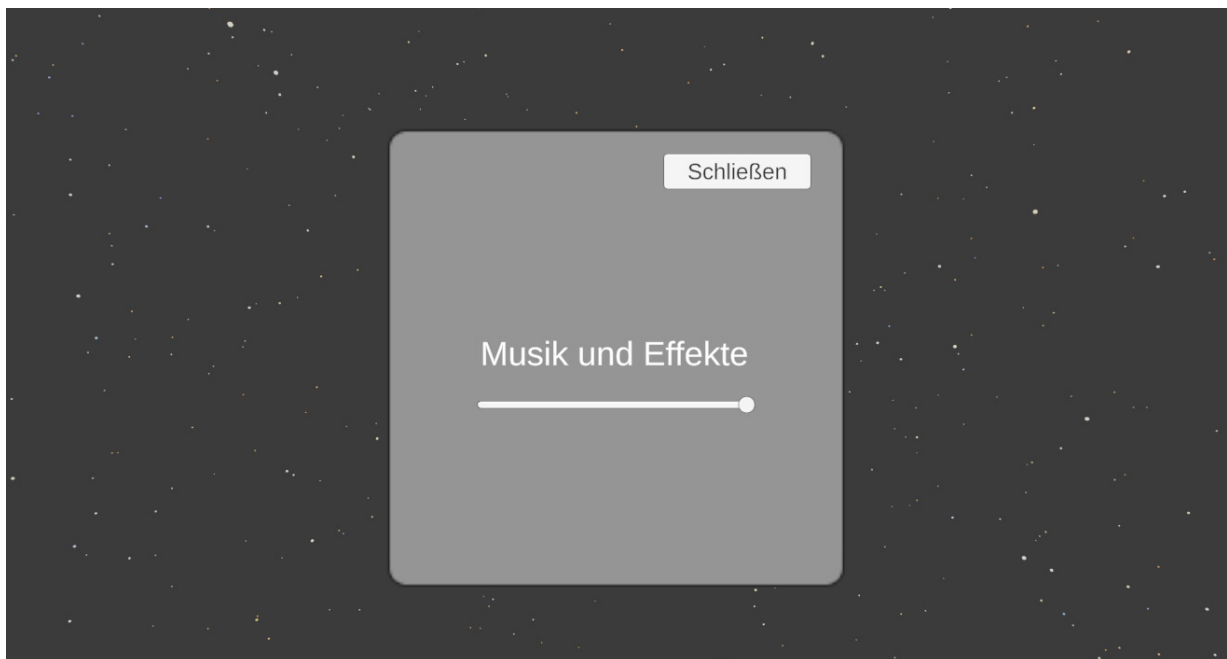
Nachdem ich das Projekt erstellt hatte, funktionierte jedoch nicht alles einwandfrei. Plötzlich konnte man mit dem Spielerobjekt nicht mehr richtig interagieren. Ich wusste nicht weiter und habe Hilfe von Kommilitonen bekommen, um meinen Fehler zu erkennen.

Projektdurchführung:

Hauptmenü/Anfangsszene:

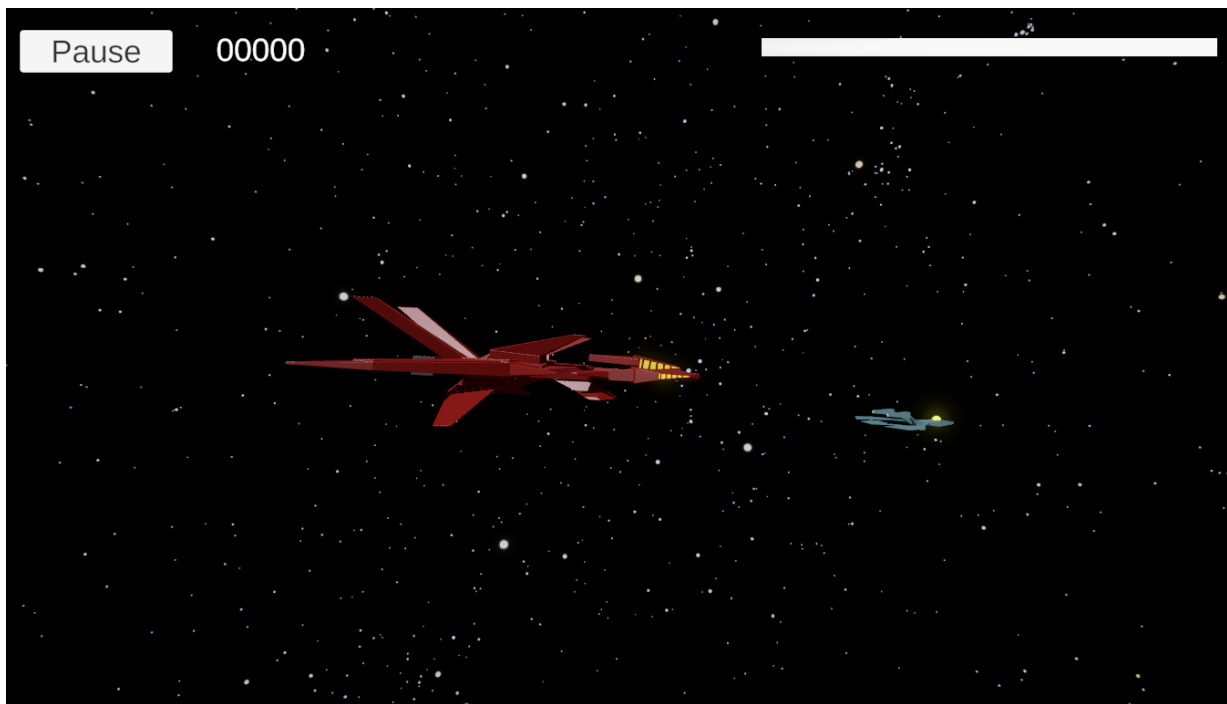


Musik und Effekte sind unter Einstellung:



Projektdurchführung:

Hauptszene:

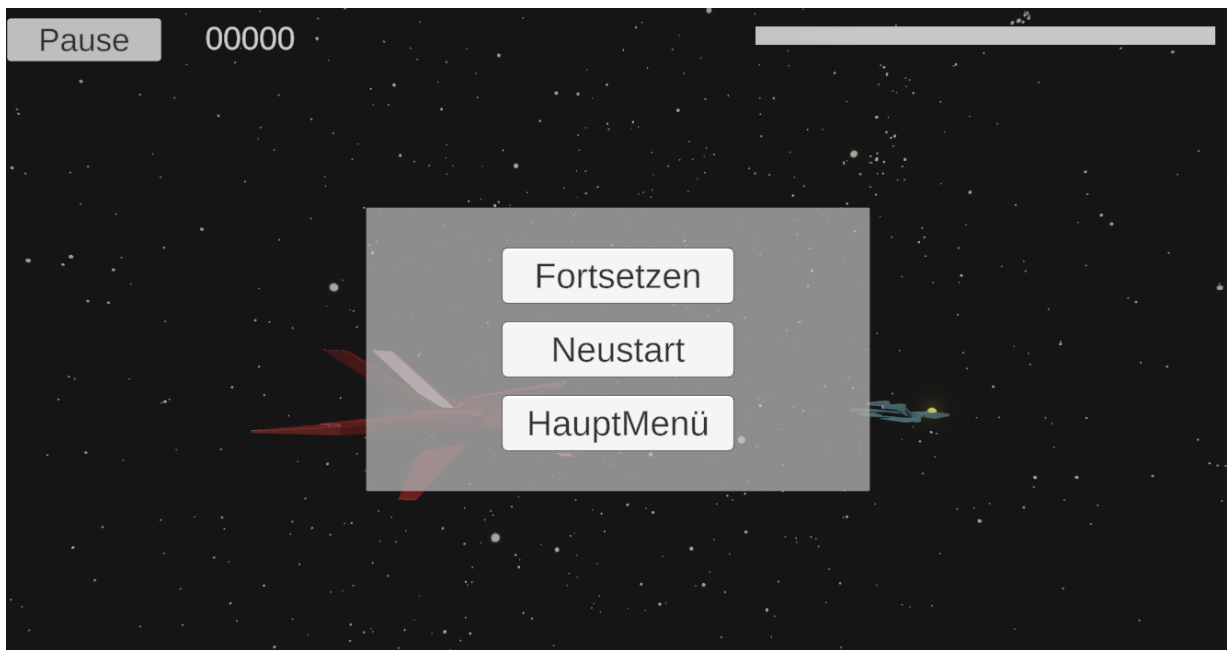


Hauptszene im Spiel/ Punkte werden gezählt:



Projektdurchführung:

Pausenmenü:



Das Game Over Menü sieht ähnlich aus wie das Pausenmenü.

Im Folgenden werde ich ein paar C#-Codezeilen zeigen, die als Beispiele dienen sollen. Es gibt verschiedene Varianten, um sein Ziel zu erreichen. Es war herausfordernd, den Code so zum Laufen zu bringen, dass keine unerwarteten Resultate entstehen. Es gab Phasen, in denen der Code nicht immer oder gar nicht funktionierte. In diesen Fällen habe ich alternative Lösungen gefunden oder den Code umgeschrieben.

Insbesondere hat das Player Movement eine große Rolle gespielt, da es die meisten Probleme verursacht hat. Deshalb möchte ich es zeigen, um zu erklären, was meine Gedanken waren und wie es zum Endergebnis kam.

Projektdurchführung:

PlayersHealth:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

Unity-Skript (1 Objektverweis) | 3 Verweise
public class PlayerHealth : MonoBehaviour
{
    public float health;
    public float maxHealth;
    public Image healthBar;

    private bool isDead;

    public GameManagerScript gameManager;
    // Start is called before the first frame update
    Unity-Nachricht | 0 Verweise
    void Start()
    {
        maxHealth = health;
    }

    // Update is called once per frame
    Unity-Nachricht | 0 Verweise
    void Update()
    {
        healthBar.fillAmount = Mathf.Clamp(health / maxHealth, 0, 1);

        if(health <= 0 && !isDead)
        {
            isDead = true;

            gameManager.gameOver();
            Debug.Log("Dead");
        }
    }
}
```

Hier wird der Gesundheitszustand des Spielers, dem kleinen Raumschiff, festgelegt. Wenn der Spieler „isDead“ ist, wird das Game-Over-Panel angezeigt. Von dort aus kann man zum Hauptmenü zurückkehren, das Spiel Neustarten oder es beenden.

Projektdurchführung:

PlayerMovment:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

Unity-Skript (1 Objektverweis) | 0 Verweise
public class PlayerMovment : MonoBehaviour
{
    public float speed = 5f;
    public float topBoundary = 0f;
    public float BottenBondary = 0f;

    // Update is called once per frame
    Unity-Nachricht | 0 Verweise
    void Update()
    {
        /** float moveVertical = 0f;

        if (Input.GetKey(KeyCode.UpArrow) || Input.GetKey(KeyCode.W))
        {
            moveVertical = 1f;
        }

        else if (Input.GetKey(KeyCode.DownArrow) || Input.GetKey(KeyCode.S))
        {
            moveVertical = -1f;
        } */

        // Vector3 movement = new Vector3(0, 0, moveVertical) * speed * Time.deltaTime;

        //transform.Translate(movement);

        float move = Input.GetAxis("Vertical") * speed * Time.deltaTime;
        Vector3 newPositon = transform.position + new Vector3(0, move, 0);

        newPositon.y = Mathf.Clamp(newPositon.y, BottenBondary, topBoundary);

        transform.position = newPositon;
    }
}
```

In dieser Abbildung sieht man die ersten zwei Versuche, den Spieler zu bewegen. In der auskommentierten Variante habe ich vergessen, eine Begrenzung für den Spieler einzubauen, weshalb man unendlich nach oben oder unten steuern konnte. In der unteren Variante ist dies nicht mehr der Fall, aber beim Erstellen des Spiels lag das kleine Raumschiff.

Projektdurchführung:

PlayerMovment:

🔗 Unity-Nachricht | 0 Verweise

```
private void FixedUpdate()
{
    float moveVertical = Input.GetAxis("Vertical");
    Vector3 movement = new Vector3(0, moveVertical, 0) * speed * Time.fixedDeltaTime;

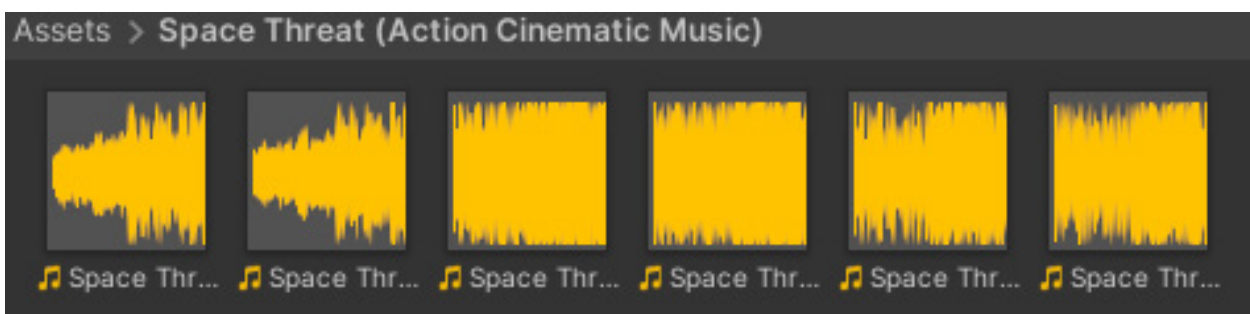
    float currentYPosition = rb.position.y;
    float currentXPosition = rb.position.x;
    float currentZPosition = rb.position.z;

    if (currentYPosition + movement.y > topBoundary)
    {
        rb.MovePosition(new Vector3(currentXPosition, topBoundary, currentZPosition));
    }
    else if (currentYPosition + movement.y < BottenBondary)
    {
        rb.MovePosition(new Vector3(currentXPosition, BottenBondary, currentZPosition));
    }
    else
    {
        rb.MovePosition(rb.position + movement);
    }
}
```

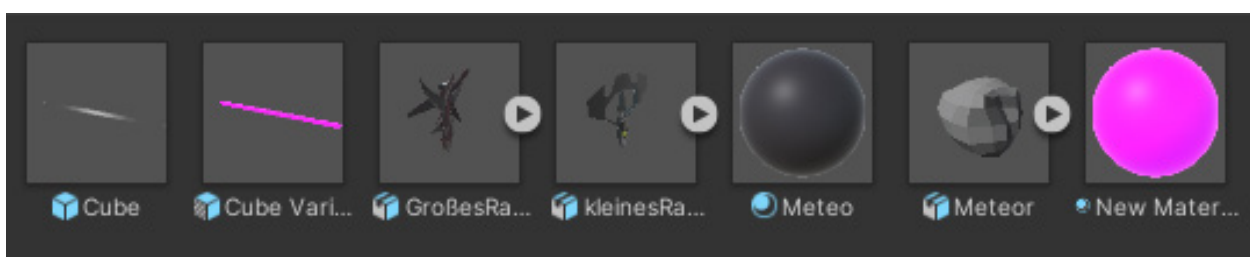
In dieser endgültigen Version des Codes wurde dann anstatt DeltaTime, fixedDeltaTime verwendet, damit das kleine Raumschiff nach dem Erstellen des Spiels keine Lags mehr hat.

Projektdurchführung:

Verwendete Assets die von Unitys Assetsstore kommen:



Selbst erstellte Assets:



Leider konnte ich nicht alle Assets, die ich erstellt habe, verwenden, da es manchmal schwierig war, diese so einzubauen, dass sie gut in die Szene passten.

Fazit:

Es hat Spaß gemacht, ein Spiel zu erstellen, auch wenn nicht alles von Anfang an geklappt hat. Am Ende habe ich dennoch ein spielbares Spiel erstellt. Man muss jedoch sagen, dass es noch Verbesserungspotenzial gibt. Es gibt viele Möglichkeiten, das Spiel weiter auszubauen, wie zum Beispiel Meteoriten im Hintergrund oder eine Anfangsszene, die zeigt, wie es zu dieser Situation gekommen ist.

Insgesamt hat es mir sehr geholfen, eigenen Code zu schreiben und mich intensiv damit auseinanderzusetzen. Ich hoffe, dass ich weiterhin an solchen Projekten arbeiten kann.

Quellen:

Assets Store:

Real Stars Skybox Lite:

<https://assetstore.unity.com/packages/3d/environments/sci-fi/real-stars-skybox-lite-116333#content>

Sci-Fi Small Sound Pack:

<https://assetstore.unity.com/packages/audio/sound-fx/sci-fi-small-sound-pack-262068#content>

Space Threat (Free Action Music):

<https://assetstore.unity.com/packages/audio/music/space-threat-free-action-music-205935>

Alle anderen Assets wurden von Unity bereitgestellt oder wurden von mir selbst erstellt.